Experiment No. 6

Experiment Name - Predictive Analytics using Linear Regression and Model Performance Evaluation

Steps

NAME : Atul Krishna

COURSE : B.Tech CSE

SECTION - B

SUBJECT : Mastery in Data Analytics and Visualizations and Career Advancement

ROLL NO. : 14

CU ID : CU24250231

1. Select a suitable dataset such as the House Price Prediction Dataset, Advertising Sales Dataset, or California Housing Dataset.

For this experiment, we use the **House Price Prediction Dataset**.

The dataset contains information about different residential properties and their characteristics. It can be used to develop a Linear Regression model for predicting the price of a house.

The dataset contains information such as:

- Area of the house
- Number of Bedrooms
- Number of Bathrooms
- Number of Floors
- Year Built
- Location
- Condition of the House
- Garage Availability
- House Price

In this experiment, **Price** is considered the dependent (target) variable, while the other relevant property characteristics are considered independent (input) variables.

The objective is to use these property features to build a **Linear Regression model** that can predict the price of a house.

2. Import the required libraries including Pandas, NumPy, Matplotlib, and Scikit-learn, and load the dataset.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files
uploaded = files.upload()  # Select the House Price Prediction Dataset CSV

import os
print(os.listdir())  # confirm the exact filename

print("Dataset loaded successfully!")

df = pd.read_csv("House Price Prediction Dataset.csv", encoding="latin1")

print("Dataset loaded successfully!")
print("Shape of dataset:", df.shape)
```

```
Choose Files House Pric… Dataset.csv
House Price Prediction Dataset.csv(text/csv) - 91297 bytes, last modified: 9/6/2026 - 100% done
Saving House Price Prediction Dataset.csv to House Price Prediction Dataset (2).csv
['.config', 'House Price Prediction Dataset (1).csv', 'House Price Prediction Dataset.csv', 'House Price Prediction Dataset
Dataset loaded successfully!
Dataset loaded successfully!
Shape of dataset: (2000, 10)
```

```
# Display first five rows
df.head()
```

	Id	Area	Bedrooms	Bathrooms	Floors	YearBuilt	Location	Condition	Garage	Price
0	1	1360	5	4	3	1970	Downtown	Excellent	No	149919
1	2	4272	5	4	3	1958	Downtown	Excellent	No	424998
2	3	3592	2	2	3	1938	Downtown	Good	No	266746
3	4	966	4	2	2	1902	Suburban	Fair	Yes	244020
4	5	4926	1	4	2	1975	Downtown	Fair	Yes	636056

Next steps: Generate code with df | New interactive sheet

3. Perform basic data preprocessing by handling missing values, encoding categorical variables (if any), and selecting relevant features.

```
# 1. Check for missing values
print("Missing values in each column:")
print(df.isnull().sum())

# Fill missing numerical values with median
numerical_columns = ['Area', 'Bedrooms', 'Bathrooms', 'Floors', 'YearBuilt']

for column in numerical_columns:
    df[column] = df[column].fillna(df[column].median())

# Fill missing categorical values with mode
categorical_columns = ['Location', 'Condition', 'Garage']

for column in categorical_columns:
    df[column] = df[column].fillna(df[column].mode()[0])

print("\nMissing values after handling:")
print(df.isnull().sum())


# 2. Select relevant features
X = df[['Area', 'Bedrooms', 'Bathrooms', 'Floors',
        'YearBuilt', 'Location', 'Condition', 'Garage']]

# Target variable
y = df['Price']

print("\nSelected Features:")
print(X.columns.tolist())

print("\nTarget Variable:")
print("Price")


# 3. Encode categorical variables
X = pd.get_dummies(
    X,
    columns=['Location', 'Condition', 'Garage'],
    drop_first=True
)

print("\nData after encoding categorical variables:")
display(X.head())

print("\nShape of processed data:", X.shape)
```

```
Missing values in each column:
Id           0
Area         0
Bedrooms     0
Bathrooms    0
Floors       0
YearBuilt    0
Location     0
Condition    0
Garage       0
Price        0
dtype: int64

Missing values after handling:
Id           0
Area         0
Bedrooms     0
Bathrooms    0
Floors       0
YearBuilt    0
Location     0
Condition    0
Garage       0
Price        0
dtype: int64

Selected Features:
['Area', 'Bedrooms', 'Bathrooms', 'Floors', 'YearBuilt', 'Location', 'Condition', 'Garage']

Target Variable:
Price

Data after encoding categorical variables:
```

	Area	Bedrooms	Bathrooms	Floors	YearBuilt	Location_Rural	Location_Suburban	Location_Urban	Condition_Fair	Condition
0	1360	5	4	3	1970	False	False	False	False	
1	4272	5	4	3	1958	False	False	False	False	
2	3592	2	2	3	1938	False	False	False	False	
3	966	4	2	2	1902	False	True	False	True	
4	4926	1	4	2	1975	False	False	False	True	

```
Shape of processed data: (2000, 12)
```

4. Split the dataset into training and testing sets using an appropriate train-test ratio (e.g., 80:20).

```
# Splitting dataset into training and testing sets
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (1600, 12)
Testing data: (400, 12)
```

5. Train a Linear Regression model using the training dataset.

```
# Training Linear Regression model
from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X_train, y_train)

print("Linear Regression model trained successfully!")
```

```
Linear Regression model trained successfully!
```

6. Use the trained model to predict values for the testing dataset.

```
# Predicting house prices for testing data
y_pred = model.predict(X_test)
```

```
print("Predictions generated successfully!")
print(y_pred[:10])
```

```
Predictions generated successfully!
[521988.22189839 549119.31196651 487101.22235594 539752.7439933
 553242.24872512 521375.92025826 523320.18080583 578133.64353335
 545899.64738458 577368.69940644]
```

```
# Comparing actual and predicted prices
comparison = pd.DataFrame({
    'Actual Price': y_test,
    'Predicted Price': y_pred
})

comparison.head(10)
```

	Actual Price	Predicted Price
1860	514764	521988.221898
353	694256	549119.311967
1333	66375	487101.222356
905	650243	539752.743993
1289	223285	553242.248725
1273	468127	521375.920258
938	513002	523320.180806
1731	911525	578133.643533
65	723265	545899.647385
1323	339416	577368.699406

Next steps: Generate code with comparison | New interactive sheet

7. Visualize the regression line and compare the actual and predicted values using appropriate plots.

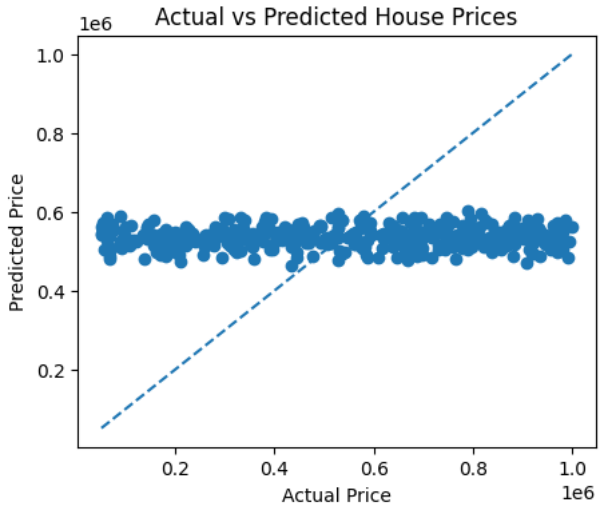
```
# Actual vs Predicted values with reference line
plt.figure(figsize=(5, 4))

plt.scatter(y_test, y_pred)

plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    linestyle='--'
)

plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted House Prices")

plt.show()
```



8. Evaluate the model using performance metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 Score.

```
# (a): Mean Absolute Error
from sklearn.metrics import mean_absolute_error
mae = mean_absolute_error(y_test, y_pred)
print("Mean Absolute Error (MAE):", mae)

# (b): Mean Squared Error
from sklearn.metrics import mean_squared_error
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error (MSE):", mse)

# (c): Root Mean Squared Error
rmse = np.sqrt(mse)
print("Root Mean Squared Error (RMSE):", rmse)

# (d): R² Score
from sklearn.metrics import r2_score
r2 = r2_score(y_test, y_pred)
print("R² Score:", r2)
```

```
Mean Absolute Error (MAE): 243241.97758826384
Mean Squared Error (MSE): 78321466146.0328
Root Mean Squared Error (RMSE): 279859.72583784326
R² Score: -0.006717808430749761
```

9. Analyze the regression coefficients to understand the impact of each independent variable on the target variable

```
# Analyze regression coefficients
coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_
})

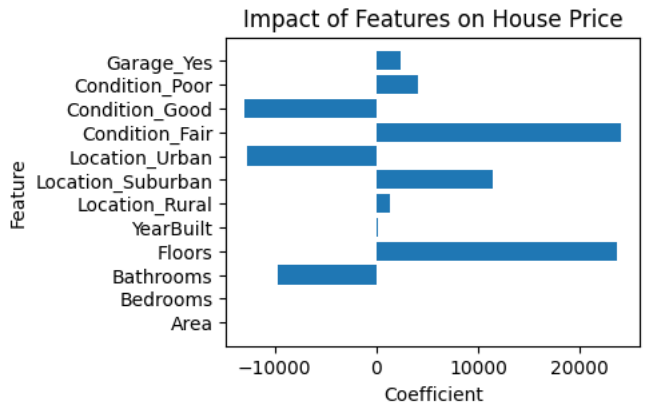
print(coefficients)
```

```
              Feature   Coefficient
0                Area     -0.575754
1            Bedrooms     76.784827
2           Bathrooms  -9662.248234
3              Floors  23727.983633
4           YearBuilt    117.613885
5      Location_Rural   1317.542020
6   Location_Suburban  11511.989815
7      Location_Urban -12718.918912
8      Condition_Fair  24083.307846
9      Condition_Good -12941.044835
10     Condition_Poor   4073.268930
11         Garage_Yes   2373.530579
```

```
# Sort coefficients
coefficients.sort_values(
    by='Coefficient',
    ascending=False
)
```

	Feature	Coefficient
8	Condition_Fair	24083.307846
3	Floors	23727.983633
6	Location_Suburban	11511.989815
10	Condition_Poor	4073.268930
11	Garage_Yes	2373.530579
5	Location_Rural	1317.542020
4	YearBuilt	117.613885
1	Bedrooms	76.784827
0	Area	-0.575754
2	Bathrooms	-9662.248234
7	Location_Urban	-12718.918912
9	Condition_Good	-12941.044835

```
# Plot regression coefficients
plt.figure(figsize=(4, 3))
plt.barh(coefficients['Feature'], coefficients['Coefficient'])
plt.xlabel("Coefficient")
plt.ylabel("Feature")
plt.title("Impact of Features on House Price")
plt.show()
```



10. Interpret the results and discuss the suitability of Linear Regression for the selected prediction problem.

The Linear Regression model was successfully trained to predict house prices using features such as Area, Bedrooms, Bathrooms, Floors, Year Built, Location, Condition, and Garage.

The model performance was evaluated using MAE, MSE, RMSE, and R^2 Score. Lower MAE, MSE, and RMSE values indicate smaller prediction errors, while an R^2 Score closer to 1 indicates better model performance.

The regression coefficients help to understand the impact of each independent variable on the house price. A positive coefficient indicates a positive relationship with price, while a negative coefficient indicates a negative relationship, keeping other variables constant.

Linear Regression is suitable for this problem because the target variable, House Price, is continuous and the model is simple and easy to interpret. However, house prices may also depend on complex and non-linear relationships. Therefore, more advanced models such as Decision Tree or Random Forest can be considered if higher prediction accuracy is required.

Conclusion: Linear Regression provides a simple and interpretable baseline model for house price prediction and is suitable for understanding the relationship between property features and price.

Questions and Answers

1. What is Predictive Analytics, and how is it used in real-world applications?

Predictive Analytics is the process of using historical data, statistical techniques, and machine learning algorithms to predict future outcomes. It is used in house price prediction, sales forecasting, fraud detection, customer behavior prediction, and demand forecasting.

2. Explain the working principle of the Linear Regression algorithm.

Linear Regression finds the relationship between independent variables and a continuous dependent variable by fitting a best-fit line to the data. The model predicts the target value using the input features and their coefficients. It tries to minimize the difference between actual and predicted values.

3. Differentiate between dependent and independent variables with suitable examples.

An independent variable is an input feature used to make a prediction, while a dependent variable is the output or target that we want to predict.

For example, in house price prediction, Area, Bedrooms, Bathrooms, Floors, and Year Built are independent variables, while House Price is the dependent variable.

4. Why is it necessary to split the dataset into training and testing sets?

The dataset is split into training and testing sets to evaluate how well the model performs on unseen data. The training set is used to train the model, while the testing set is used to evaluate its performance. This helps determine whether the model can generalize to new data.

5. What is the significance of the R^2 Score in regression analysis?

R^2 Score measures how well the independent variables explain the variation in the dependent variable. An R^2 Score closer to 1 indicates a better model fit, while a score closer to 0 indicates a weaker model fit.

6. Differentiate between MAE, MSE, and RMSE. Which metric is more sensitive to large prediction errors?

- MAE (Mean Absolute Error) calculates the average absolute difference between actual and predicted values.
- MSE (Mean Squared Error) calculates the average of squared prediction errors.
- RMSE (Root Mean Squared Error) is the square root of MSE.

MSE and RMSE are more sensitive to large prediction errors because the errors are squared.

7. What assumptions should be satisfied before applying Linear Regression?

The main assumptions of Linear Regression are:

1. There should be a linear relationship between the variables.
2. Observations should be independent.
3. The variance of errors should be constant.
4. Residuals should be approximately normally distributed.
5. There should be no strong multicollinearity among independent variables.

8. How can overfitting and underfitting affect the performance of a regression model?

- Overfitting occurs when a model learns the training data too closely and performs poorly on new data.
- Underfitting occurs when a model is too simple and cannot capture the important patterns in the data.
- A good regression model should perform well on both training and testing data.

9. Mention any three real-world applications of Linear Regression in business or industry.

Three real-world applications of Linear Regression are:

1. House price prediction
2. Sales and revenue forecasting
3. Demand prediction

It can also be used for cost estimation and financial forecasting.

10. How can feature selection improve the accuracy and interpretability of a predictive model?

Feature selection removes irrelevant or redundant features from the dataset. It can improve model accuracy, reduce overfitting, decrease computational complexity, and make the model easier to understand and interpret.

For example, in house price prediction, important features such as Area, Bedrooms, Bathrooms, and Location can be selected for prediction.